Ejercicio práctico 1.5

«Pedidos bajo control»

Nombre del estudiante	Samantha Quinche
Curso y paralelo	Tercero A Matutina
Fecha	7/11/2026
Git Hub	https://github.com/ssquinche-oss/Actividad5.git

Asignatura	Herramientas informáticas para el despliegue de diagramas
Unidad	1. Principios de la arquitectura web en el diseño y desarrollo de aplicaciones backend
Tema	1.5 Arquitectura de software: capas y arquitectura hexagonal
Docente	Jhostin Vacacela
Duración	65 minutos en clase
Modalidad	Individual o en parejas

Objetivo

Ubicar cada regla de un sistema de pedidos en la capa que le corresponde y separar su acceso a datos mediante un puerto.

Situación

Una papelería universitaria recibe encargos de cuadernos, lápices y otros útiles. Cada encargo es un pedido y pasa por tres estados: PENDIENTE cuando se registra, ENVIADO cuando sale de la papelería y RECIBIDO cuando el cliente lo confirma con un código.

El equipo escribió el backend en Kotlin con Spring Boot y PostgreSQL. Tu trabajo es leerlo, completarlo, corregirlo y proponer cómo aislar su acceso a datos.

El sistema

Hay dos tablas. En productos se guarda el nombre, la descripción y el stock. En pedidos se guarda, por cada producto pedido, el código del pedido, el nombre del cliente, el nombre del producto, la cantidad, el estado y el código de verificación. Un pedido con varios productos ocupa varias filas que comparten el mismo código de pedido; no existe tabla intermedia.

Los endpoints son solo cuatro:

Método	Ruta	Qué hace
POST	/productos	Registra un producto con su stock
POST	/pedidos	Crea un pedido y devuelve su código de verificación
PATCH	/pedidos/{codigo}/enviar	Pasa el pedido de PENDIENTE a ENVIADO
PATCH	/pedidos/{codigo}/recibir	Pasa el pedido de ENVIADO a RECIBIDO si el código es correcto

Las reglas de negocio son estas, y todas van en la capa de servicio:

Regla	Descripción	Si se incumple
R1	Un cliente no puede hacer un pedido nuevo si ya tiene otro PENDIENTE	409 Conflict
R2	No se puede pedir más cantidad de la que hay en stock	409 Conflict
R3	El producto pedido debe existir	404 Not Found
R4	El pedido solo avanza de PENDIENTE a ENVIADO y de ENVIADO a RECIBIDO	409 Conflict
R5	Para marcar RECIBIDO se necesita el código de verificación del pedido	400 Bad Request
R6	Al crear el pedido, el stock de cada producto baja	—
R7	El código de verificación lo genera el sistema con un número aleatorio de seis dígitos	—

Instrucciones generales

1. Trabaja solo o con un compañero. Si es en pareja, escriban los dos nombres arriba.
2. Responde en los recuadros. El código se puede escribir a mano o en computadora; no importa que no compile, importa que se entienda la idea.
3. Entrega este documento en PDF o impreso al terminar los 65 minutos.
4. Si usas inteligencia artificial, declara la herramienta y para qué la usaste en la Parte E. Como referencia se acepta hasta un 30 % de contenido posiblemente generado por IA y un 10 % de similitud.

Reparto de puntos:

Parte	Qué se hace	Máximo	Obtenido
A	Leer y predecir	20	
B	Completar	25	
C	Detectar y corregir errores	25	
D	Diseñar y escribir	25	
E	Reflexión y declaración de IA	5	
Total		100	

Parte A. Leer y predecir (20 puntos)

El sistema arranca así: el producto Cuaderno tiene stock 10, el producto Lapiz tiene stock 5 y no hay ningún pedido. Se hacen las cinco peticiones de la tabla, una tras otra y en ese orden. Completa las cuatro columnas de cada fila. El estado que se pide es el del pedido de Ana. Cada casilla vale 1 punto.

N.º	Petición	Código HTTP	Estado del pedido de Ana	Stock de Cuaderno	Stock de Lapiz
1	POST /pedidos. Ana pide 2 Cuaderno y 3 Lapiz	201	Pendiente	8	2
2	POST /pedidos. Luis pide 9 Lapiz	409	Solicitado 9 lapices / Pendiente	8	2
3	POST /pedidos. Ana pide 1 Lapiz	409	El cliente ya tiene un pedido pendiente	8	2
4	PATCH /pedidos/{codigo}/recibir. Pedido de Ana, con su código de verificación correcto	409	Pedido enviado	8	2
5	PATCH /pedidos/{codigo}/enviar. Pedido de Ana	200	Enviado	8	2

Parte B. Completar (25 puntos)

Este es un fragmento del servicio de pedidos. Faltan cinco piezas marcadas con B1 a B5. Escribe lo que va en cada lugar. Cada acierto vale 5 puntos.

```
fun recibir(codigoPedido: String, codigoVerificacion: String): PedidoResponse {
    val filas = buscar(codigoPedido)
    if (filas.first().estado != B1) {
        throw EstadoInvalidoException("Solo un pedido ENVIADO se puede recibir")
    }
    if (filas.first().codigoVerificacion B2 codigoVerificacion) {
        throw CodigoVerificacionIncorrectoException()
    }
    filas.forEach { it.estado = B3 }
    return respuesta(filas)
}
```

```
// dentro de crear(), una vez comprobado que todos los productos existen:
pedidoPorProducto.forEach { (nombre, cantidad) ->
    val producto = encontrados.getValue(nombre)
    if (producto.stock B4 cantidad) {
        throw StockInsuficienteException(nombre, producto.stock, cantidad)
    }
}
// y más abajo, al guardar cada fila del pedido:
encontrados.getValue(nombre).stock B5 cantidad
```

Pieza	Tu respuesta	¿A qué regla corresponde?
B1	EstadoPedido.Enviadpo	Regla 4
B2	!=	Regla 5
B3	EstadoPedido.Recibido	Regla 4
B4	<	Regla 2
B5	-=	Regla 6

Parte C. Detectar y corregir errores (25 puntos)

Los tres fragmentos siguientes tienen cinco errores en total: dos en el primero, dos en el segundo y uno en el tercero. Encuéntralos. Por cada error escribe qué está mal y por qué (2 puntos) y cómo se corrige (3 puntos).

Fragmento C1. Controlador

```
@RestController
@RequestMapping("/pedidos")
class PedidoController(
    private val productos: ProductoRepository,
    private val servicio: PedidoService
) {
    @PostMapping
    fun crear(@RequestBody request: CrearPedidoRequest): PedidoCreadoResponse {
        val item = request.items.first()
        val producto = productos.findByNombre(item.producto)
            ?: throw ProductoNoExisteException(item.producto)
        if (producto.stock < item.cantidad) {
            throw StockInsuficienteException(
                item.producto, producto.stock, item.cantidad
            )
        }
        return servicio.crear(request)
    }
}
```

Fragmento C2. Servicio, método crear

```
@Transactional
fun crear(request: CrearPedidoRequest): PedidoCreadoResponse {
    if (pedidos.existsByClienteAndEstado(request.cliente, EstadoPedido.PENDIENTE)) {
        throw ClienteConPedidoPendienteException(request.cliente)
    }
    val codigoPedido = "PED-" + UUID.randomUUID().toString().take(8).uppercase()
    request.items.forEach { item ->
        val producto = productos.findByNombre(item.producto)
            ?: throw ProductoNoExisteException(item.producto)
        producto.stock -= item.cantidad
        pedidos.save(
            Pedido(codigoPedido, request.cliente, item.producto, item.cantidad,
                codigoVerificacion = "123456")
        )
    }
    // ... se arma y devuelve la respuesta
}
```

Fragmento C3. Servicio, método recibir

```
@Transactional
fun recibir(codigoPedido: String, codigoVerificacion: String): PedidoResponse {
    val filas = buscar(codigoPedido)
    if (filas.first().codigoVerificacion != codigoVerificacion) {
        throw CodigoVerificacionIncorrectoException()
    }
    filas.forEach { it.estado = EstadoPedido.RECIBIDO }
    return respuesta(filas)
}
```

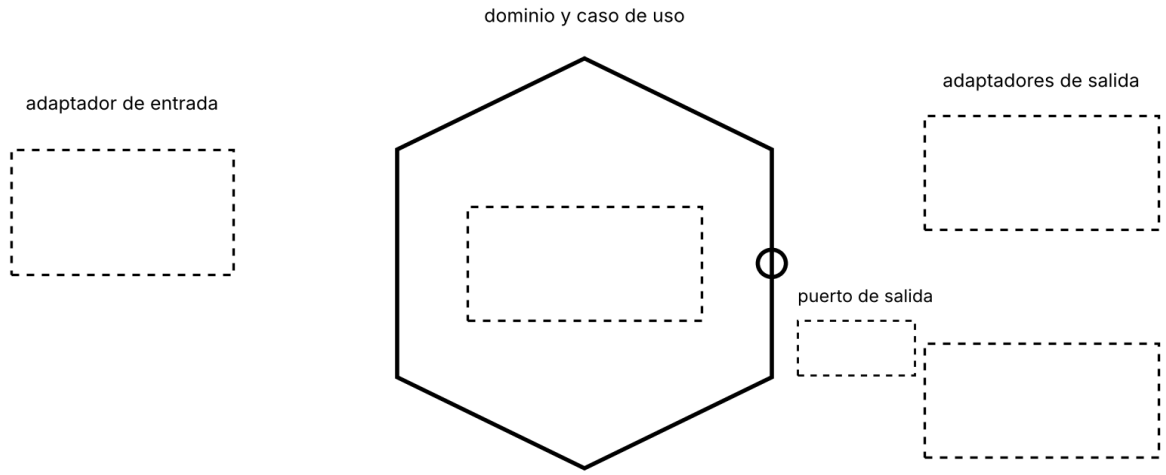
N.º	Qué está mal y por qué (2 puntos)	Cómo se corrige (3 puntos)
1	C1, controlador: accede directamente a `ProductoRepository` y valida el stock. Es una regla de negocio (R2) en la capa de entrada, cuando todas las reglas van en el servicio.	Quitar el repositorio del controlador y dejar que solo llame a `servicio.crear(request)`.
2	C1, controlador: solo revisa `request.items.first()`, así que ignora los demás productos del pedido.	Eliminar esa validación del controlador; el servicio ya recorre todos los ítems.
3	C2, servicio: el código de verificación está fijo en `"123456"`, lo que incumple R7 (debe ser aleatorio de seis dígitos).	Generarlo con `Random.nextInt(100000, 1000000).toString()`.
4	C2, servicio: descuenta el stock sin comprobar que alcance, así que puede quedar negativo. Incumple R2.	Antes de `producto.stock -= item.cantidad`, agregar `if (producto.stock < item.cantidad) throw StockInsuficienteException(item.producto, producto.stock, item.cantidad)`.
5	C3, servicio: `recibir` no comprueba que el pedido esté ENVIADO, así que un pedido PENDIENTE podría pasar a RECIBIDO. Incumple R4.	Agregar al inicio `if (filas.first().estado != EstadoPedido.ENVIADO) throw EstadoInvalidoException("Solo un pedido ENVIADO se puede recibir")`.

Parte D. Diseñar y escribir (25 puntos)

El equipo quiere probar las reglas de los pedidos sin levantar PostgreSQL. Para lograrlo, el servicio debe hablar con un puerto y no con el repositorio de Spring Data.

D1. Diagrama hexagonal (7 puntos)

En el diagrama, escribe en cada recuadro el nombre que le corresponde. Elige entre estos cinco: PedidoController, PedidoService, PedidoRepositoryPort, PedidoJpaAdapter y PedidosEnMemoria. Después dibuja dos flechas que muestren quién depende de quién: una desde el adaptador de entrada hacia el centro y otra desde un adaptador de salida hacia el puerto. Cada nombre bien ubicado vale 1 punto y cada flecha bien dirigida vale 1 punto.



No pude escribir sobre la imagen pero es:

Pedidocontroller > Pedido service > PedidoRepositoryPort < PedidoJpaAdapter

< PedidosEnMemoria

D2. El puerto (6 puntos)

Escribe en Kotlin la interfaz PedidoRepositoryPort con tres operaciones, 2 puntos cada una: preguntar si un cliente tiene un pedido pendiente (devuelve verdadero o falso), guardar una fila de pedido (recibe y devuelve un `Pedido`) y buscar todas las filas de un código de pedido (devuelve una lista de `Pedido`).

interface PedidoRepositoryPort

```
interface PedidoRepositoryPort {
    fun existePedidoPendiente(cliente: String): Boolean
    fun guardar(pedido: Pedido): Pedido
    fun buscarPorCodigo(codigoPedido: String): List<Pedido>
}
```

D3. El adaptador en memoria (9 puntos)

Escribe la clase PedidosEnMemoria que cumple ese puerto guardando las filas en una lista mutable. Cada operación bien implementada vale 3 puntos. El estado PENDIENTE es `EstadoPedido.PENDIENTE`.

class PedidosEnMemoria : PedidoRepositoryPort

```
class PedidosEnMemoria : PedidoRepositoryPort {

    private val filas = mutableListOf<Pedido>()

    override fun existePedidoPendiente(cliente: String): Boolean =
        filas.any { it.cliente == cliente && it.estado == EstadoPedido.PENDIENTE }

    override fun guardar(pedido: Pedido): Pedido {
        filas.add(pedido)
        return pedido
    }

    override fun buscarPorCodigo(codigoPedido: String): List<Pedido> =
        filas.filter { it.codigoPedido == codigoPedido }
}
```

D4. Para qué sirve (3 puntos)

En dos o tres frases: ¿qué se gana al probar las reglas de `PedidoService` con `PedidosEnMemoria` en lugar de PostgreSQL?

Pues se verifica sin consultar en postgres lo que permite probar rápidamente y hacer pruebas sin consultar o modificar el funcionamiento de la base de datos, si algo falla es debido al servicio y no la conexión a los datos.

Parte E. Reflexión y declaración de IA (5 puntos)

Reflexión (3 puntos). ¿Qué regla te costó más ubicar en su capa y por qué? Responde en dos o tres líneas.

La mas difícil el la Regla 4 porque hay que validar datos.

Declaración de uso de IA (2 puntos). Si no usaste IA, escribe «No usé IA». Si la usaste, completa la tabla.

Herramienta	Para qué la usaste	Qué verificaste por tu cuenta
Claude	Generar código debido a obtener errores en conexión en base de datos o al recibir y enviar datos al validar una operación o servicio	Funcionamiento del proyecto correctamente en postman, verificar que lo modificado por la IA no afecte mi trabajp

Rúbrica de evaluación (100 puntos)

El instrumento que fija el PEA para este tema es la rúbrica. Los puntos de cada parte se suman por casilla, pieza, error o elemento, como indica cada parte. Esta tabla describe cómo se lee el desempeño en cada criterio.

Criterio	**Máximo**	**Excelente (90 % o más)**	**Bueno (70 a 89 %)**	**Básico (40 a 69 %)**	**Insuficiente (menos de 40 %)**
A. Predicción del comportamiento	20	Prevé código HTTP, estado y stock en casi todas las peticiones	Falla en algún estado o en el stock	Acierta solo los casos simples	No relaciona las peticiones con las reglas
B. Reglas de negocio en la capa de servicio	25	Completa las cinco piezas y nombra su regla	Completa cuatro, con la regla bien nombrada	Completa dos o tres	Completa una o ninguna
C. Detección y corrección de errores	25	Encuentra los cinco y corrige indicando la capa	Encuentra cuatro	Encuentra dos o tres	Encuentra uno o ninguno
D. Diseño con puerto y adaptador	25	Diagrama, puerto, adaptador y justificación completos	Falla un detalle menor	Falta el adaptador o el puerto	Sin puerto ni adaptador
E. Reflexión y declaración de IA	5	Reflexión concreta y declaración completa	Una de las dos incompleta	Ambas incompletas	No entregada
Total	100				

Lista de verificación antes de entregar

- La tabla de la Parte A tiene sus 20 casillas llenas y cada código HTTP es uno de 201, 400, 404, 409 o 200.
- En la Parte B escribiste una regla (R1 a R7) para cada pieza.
- La Parte C tiene cinco filas completas y cada corrección dice en qué capa queda la regla.
- El diagrama de la Parte D tiene cinco nombres y dos flechas que llegan al centro.
- Declaraste el uso de IA, o escribiste que no la usaste.

Referencias

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Spring. (s. f.). *Spring Boot*. https://spring.io/projects/spring-boot

Instituto Superior Tecnológico. (2026). *Programa de estudio de asignatura: Herramientas informáticas para el despliegue de diagramas, tercer ciclo* [PEA].